openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

Running Database Migrations Safely in Kubernetes

Ilham Adi Setiawan
me@ilhamset.id

Yogyakarta | 4 October 2026

What is a migration?



Image: Migration (2023), Illumination Entertainment

Not this kind.

Although ours also travel in a fixed order, and things go wrong when one gets left behind.

Migrations, in plain words

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

The code and the database must agree on the shape of the data.

Before

id	name
1	Ani
2	Budi

`0002_add_phone.py`



a migration

After

id	name	phone
1	Ani	–
2	Budi	–

1 Versioned

Numbered files, kept in git with the code.

2 Ordered

Applied one by one, never skipped.

3 Recorded

The database remembers which ones ran.

Our initial deployment workflow

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

Before: our deploy job (simplified)

```
# no migrate job in the pipeline
deploy-staging:
  stage: deploy
  script:
    - kubectl rollout restart deployment/app
    - kubectl rollout status deployment/app
        --timeout=300s
```

What the CI job log showed

```
$ kubectl rollout restart deployment/app
deployment.apps/app restarted
$ kubectl rollout status deployment/app
deployment "app" successfully rolled out
# Job succeeded
# ...and not one line about migrations
```

Migration log: nowhere in CI. Migrations ran by hand from a pod; later via kubectl exec into the running app.

Problems we faced

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

1 Invisible failures

A migration could fail while the pipeline stayed green. CI/CD had no signal.

2 Debugging in the cluster

kubectl get pods, kubectl logs, guessing which pod ran it, if it still existed.

3 Deploys ran ahead

The application rolled out before the schema change was fully applied.

4 Inconsistent environments

Staging and production ended up with different schemas and different behaviour.

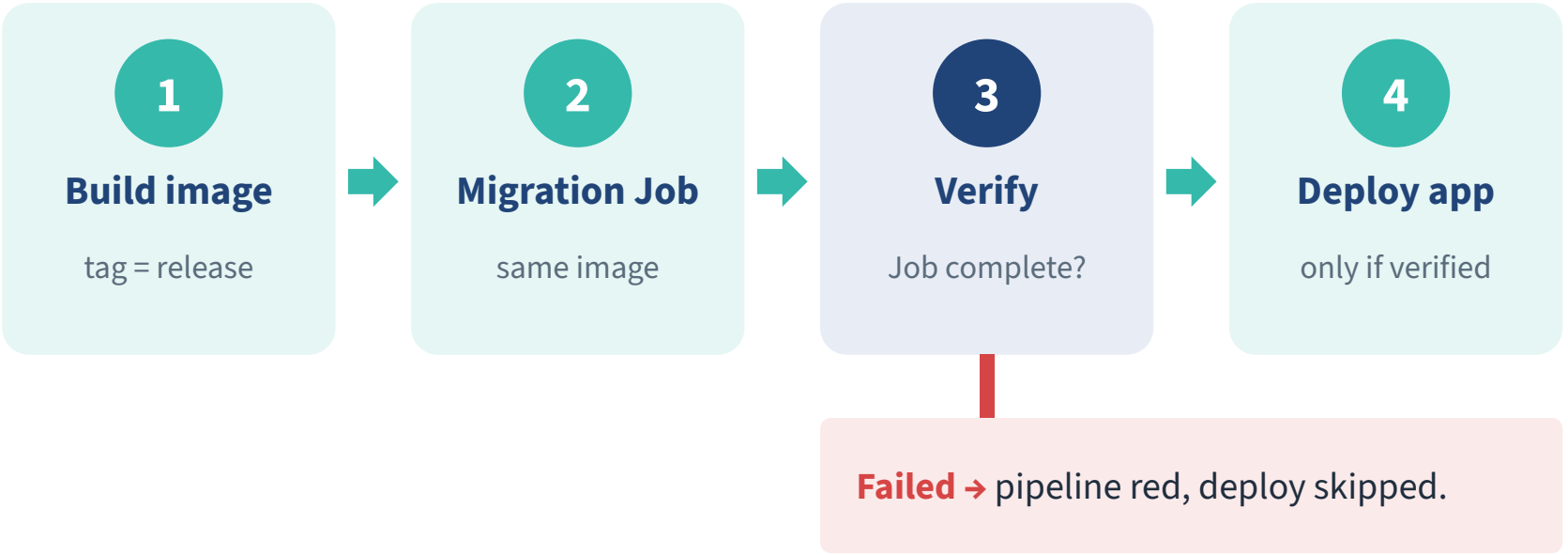
How does the pipeline know the migration actually worked?

What we needed to change

1	**Explicit**	Migration is its own stage, not a side effect of startup or deploy.
2	**Clear status**	Success or failure is unambiguous.
3	**Accessible logs**	Migration output is readable from the pipeline itself.
4	**A real gate**	A failed migration stops the deployment.
5	**Right version**	Migration code comes from the exact release being deployed.

A first-class CI/CD step

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4



Decoupled from application startup and deployment, but still part of the release pipeline, with its own stage and controls.

Anatomy of a migration Job


```
apiVersion: batch/v1
kind: Job
metadata:
  name: app-migrate-${CI_PIPELINE_ID}
spec:
  backoffLimit: 0
  activeDeadlineSeconds: 1200
  ttlSecondsAfterFinished: 300
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: migrate
        image: app:${RELEASE_TAG}
        command: ["python", "manage.py",
                  "migrate", "--noinput"]
        envFrom:
        - secretRef: {name: app-db}
```

1. **Release image**
   The exact tag that will be deployed.
2. **migrate command**
   One Job, one run, one result.
3. **DB config via Secret**
   Same source as the app's Deployment.
4. **Time limit**
   activeDeadlineSeconds: a hang becomes a failure.
5. **Retry by choice**
   backoffLimit: 0, never retry blindly.

Connecting Job and pipeline

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

Before · wait for "complete"

```
- kubectl wait job/$JOB
    --for=condition=complete
    --timeout=1200s
    || (kubectl logs job/$JOB && exit 1)
- kubectl logs job/$JOB
```

CI log when the migration fails

```
$ kubectl wait --for=condition=complete ...
# Job already Failed, but no output
# still waiting... up to 20 minutes
# ttlSecondsAfterFinished: 300 may
#   delete the Job and its logs first
```

After · poll both outcomes

```
while [ $ELAPSED -lt $TIMEOUT ]; do
  COMPLETE=$(kubectl get job ... Complete)
  FAILED=$(kubectl get job ... Failed)
  if [ "$FAILED" = "True" ]; then
    echo "MIGRATION FAILED"
    echo "Reason: ${REASON}"
    kubectl logs job/$JOB_NAME; exit 1
```

CI log when the migration fails

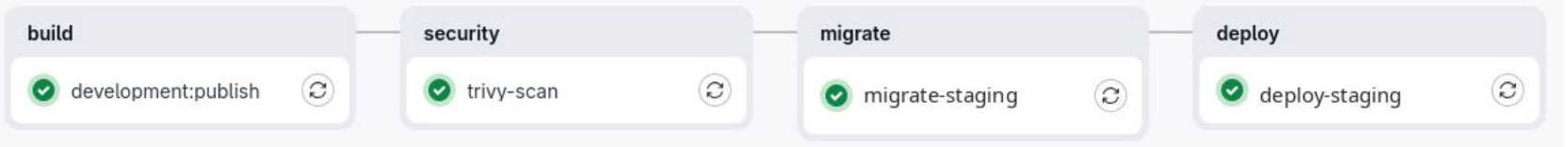
```
Waiting for migration job...
MIGRATION FAILED
Reason: Job has reached the
  specified backoff limit
<full migrate output> ... exit 1
```

Same Job, same image. Only the way CI waits changed, and the log became visible.

A passing migration in CI

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

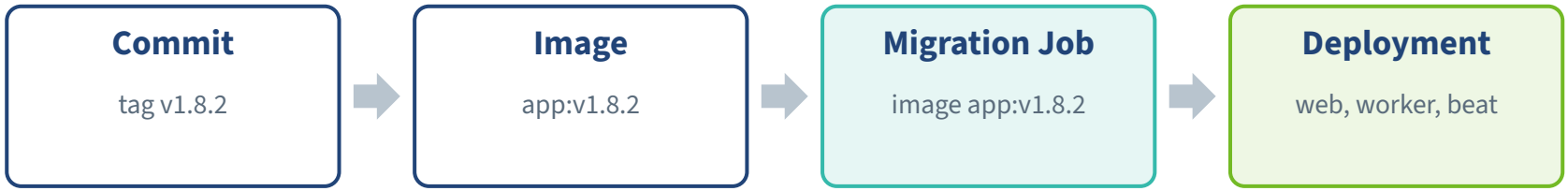
GitLab pipeline: every stage green, deploy ran after migrate



Migrate job log: the Job's output, streamed into CI

```
190  07:45:57  Operations to perform:
191  07:45:57    Apply all migrations: tunjangan
192  07:45:57  Running migrations:
193  07:45:57    Applying tunjangan.0062_add_is_kamad_to_ajuan_skmt... OK
194  07:45:57    Applying tunjangan.0064_merge_20260805_1300... OK
195  07:45:57  $ kubectl delete job/app-migrate-${CI_PIPELINE_ID} --namespace=${MIGRATE_NAMESPACE} || true
196  07:45:57  job.batch "app-migrate-69178" deleted from staging namespace
197  07:45:57  Running after_script
198  07:45:58  Running after script...
199  07:45:58  $ rm -f /tmp/ca.crt
200  07:45:58  Cleaning up project directory and file based variables
201  07:45:58  Job succeeded
```

Run the exact version you deploy



Do

- One unique tag (or digest) for Job and Deployment
- Build once, promote the same image
- Migration files come from that image

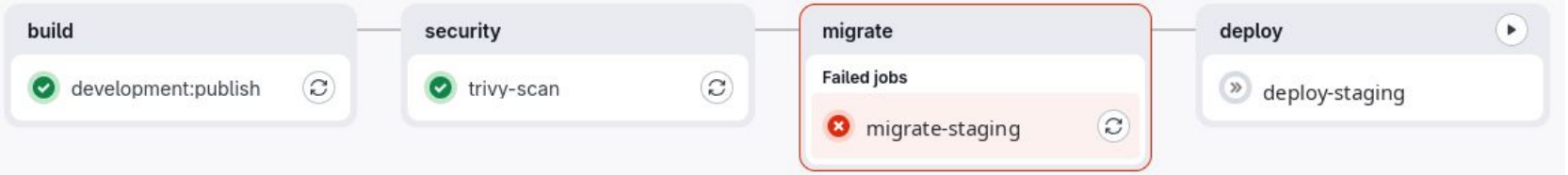
Avoid

- Mutable tags like :latest
- kubectl exec into the running app: it still has the old image
- Building a separate image just to migrate

Failure handling and debugging

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

GitLab pipeline: migrate failed, deploy was skipped



Migrate job log: the error is right there

```
324  07:18:39    File "/app/.venv/lib/python3.13/site-packages/pymysql/err.py", line 154, in raise_mysql_exception
325  07:18:39      raise errorclass(errno, errval, sqlstate=sqlstate)
326  07:18:39  django.db.utils.OperationalError: (1060, "Duplicate column name 'is_kamad'")
327  07:18:39    Applying tunjangan.0062_add_is_kamad_to_ajuan_skmt...==========================================
328  07:18:39  Running after_script
329  07:18:39  Running after script...
330  07:18:39  $ rm -f /tmp/ca.crt
331  07:18:39  Cleaning up project directory and file based variables
332  07:18:40  ERROR: Job failed: command terminated with exit code 1
```

Read the error
1060: duplicate column

Check the real schema
DDL may be half-applied

Fix, then re-run
make the migration idempotent

What Jobs do, and don't, solve

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

Jobs do

- Run the migration once, in isolation
- Give a clear Complete / Failed status
- Keep logs reachable from CI
- Let you choose retries and deadlines

Jobs don't

- **Prevent database locks**
  → short lock timeout + maintenance window
- **Stop two pipelines migrating at once**
  → interruptible: false + resource_group
- **Keep old code when deploy is skipped**
  → unique image tag per release, never :latest

Also not solved: schema changes compatible with old code. Next step: expand–contract

Expand
add new, keep old

→

Migrate
deploy code, backfill

→

Contract
drop old, later

Lessons learned

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

01 **Visible and controllable**

Migration is a stage you can see, not a hidden side effect.

02 **Failure blocks deployment**

A failed migration stops the release, not the users.

03 **Debugging from the pipeline**

Status and logs are in CI; no cluster archaeology.

04 **Consistent versions**

Migration code matches the release being deployed.

Deployment success must depend on verifiable migration success.

Thank You

Ilham Adi Setiawan



openSUSE®
Asia Summit 2026